

trial_2

June 7, 2025

Trial 2 : Variant 1: Multi-Scale + Dilation (Wide Receptive Field) Uses different dilation rates in combination with kernel sizes.

```
[1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
#from tensorflow.keras.models import Sequential
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Conv1D, BatchNormalization, Dropout,
    ↳Concatenate, Flatten, Dense
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, confusion_matrix,
    ↳roc_auc_score, roc_curve
```

1 load & Split data

```
[ ]: # Load feature and target data
features = pd.read_csv(r"D:\JN\data set\created dataset\ECG PCG_
    ↳feature(128+128=256).csv", header=None, skiprows=[0])
targets = pd.read_csv(r'D:\JN\data set\data from web\Reference_aug.csv',
    ↳header=None)

print(features.shape)
print(targets.shape)

# Extract target column
y = targets.iloc[:, -1] # Selecting the last column
X = features # All feature columns

# Using .apply() method
y = y.apply(lambda x: max(x, 0))

# Split data into training and testing sets
Xtrain, Xtest, Ytrain, Ytest = train_test_split(X, y, test_size=0.2,
    ↳random_state=42)
```

```

# Standardize features
scaler = StandardScaler()
Xtrain_scaled = scaler.fit_transform(Xtrain)
Xtest_scaled = scaler.transform(Xtest)

# Reshape data for Conv1D input
Xtrain_reshaped = Xtrain_scaled.reshape((Xtrain_scaled.shape[0], Xtrain_scaled.
    ↪shape[1], 1))
Xtest_reshaped = Xtest_scaled.reshape((Xtest_scaled.shape[0], Xtest_scaled.
    ↪shape[1], 1))

```

2 Multi-Scale Convolutional Neural Network (MSCNN) ¶

```

[3]: from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Conv1D, BatchNormalization, Dropout,
    ↪Concatenate, Flatten, Dense

input_layer = Input(shape=(256, 1))

# Multi-scale convolutions with different dilation rates
conv3 = Conv1D(16, 3, dilation_rate=1, padding='same',
    ↪activation='relu')(input_layer)
conv5 = Conv1D(16, 5, dilation_rate=2, padding='same',
    ↪activation='relu')(input_layer)
conv7 = Conv1D(16, 7, dilation_rate=3, padding='same',
    ↪activation='relu')(input_layer)

merged = Concatenate()([conv3, conv5, conv7])
merged = BatchNormalization()(merged)
merged = Dropout(0.4)(merged)

# Continue with normal convolution layers
conv = Conv1D(32, 3, activation='relu', padding='same')(merged)
conv = BatchNormalization()(conv)
conv = Dropout(0.4)(conv)
conv = Conv1D(16, 3, activation='relu', padding='same')(conv)

# Dense layers
flattened = Flatten()(conv)
dense = Dense(128, activation='relu')(flattened)
dense = Dense(64, activation='relu')(dense)
output = Dense(1, activation='sigmoid')(dense)

model = Model(inputs=input_layer, outputs=output)

```

```
model.compile(optimizer='adam', loss='binary_crossentropy',  
              metrics=['accuracy'])
```

```
[4]: # Train model  
history = model.fit(Xtrain_resaped, Ytrain, epochs=60, batch_size=32,  
                  validation_split=0.2)
```

```
Epoch 1/60  
40/40      8s 66ms/step -  
accuracy: 0.5245 - loss: 0.8247 - val_accuracy: 0.5665 - val_loss: 0.6796  
Epoch 2/60  
40/40      2s 42ms/step -  
accuracy: 0.6419 - loss: 0.6320 - val_accuracy: 0.5348 - val_loss: 0.6743  
Epoch 3/60  
40/40      2s 53ms/step -  
accuracy: 0.6582 - loss: 0.6023 - val_accuracy: 0.6108 - val_loss: 0.6599  
Epoch 4/60  
40/40      2s 59ms/step -  
accuracy: 0.7210 - loss: 0.5525 - val_accuracy: 0.6392 - val_loss: 0.6385  
Epoch 5/60  
40/40      2s 52ms/step -  
accuracy: 0.7055 - loss: 0.5483 - val_accuracy: 0.6614 - val_loss: 0.6175  
Epoch 6/60  
40/40      2s 49ms/step -  
accuracy: 0.7423 - loss: 0.5075 - val_accuracy: 0.5918 - val_loss: 0.7100  
Epoch 7/60  
40/40      2s 53ms/step -  
accuracy: 0.7397 - loss: 0.5328 - val_accuracy: 0.6772 - val_loss: 0.5797  
Epoch 8/60  
40/40      2s 57ms/step -  
accuracy: 0.7731 - loss: 0.4562 - val_accuracy: 0.6899 - val_loss: 0.5824  
Epoch 9/60  
40/40      2s 54ms/step -  
accuracy: 0.8143 - loss: 0.4115 - val_accuracy: 0.6962 - val_loss: 0.5948  
Epoch 10/60  
40/40      2s 52ms/step -  
accuracy: 0.8165 - loss: 0.3914 - val_accuracy: 0.6930 - val_loss: 0.6685  
Epoch 11/60  
40/40      2s 49ms/step -  
accuracy: 0.8313 - loss: 0.3637 - val_accuracy: 0.7089 - val_loss: 0.6110  
Epoch 12/60  
40/40      2s 48ms/step -  
accuracy: 0.8204 - loss: 0.3858 - val_accuracy: 0.7278 - val_loss: 0.6365  
Epoch 13/60  
40/40      2s 47ms/step -  
accuracy: 0.8627 - loss: 0.3238 - val_accuracy: 0.7152 - val_loss: 0.6711  
Epoch 14/60  
40/40      2s 53ms/step -
```

accuracy: 0.8761 - loss: 0.2970 - val_accuracy: 0.7120 - val_loss: 0.6395
 Epoch 15/60
 40/40 3s 62ms/step -
 accuracy: 0.9006 - loss: 0.2369 - val_accuracy: 0.7310 - val_loss: 0.6861
 Epoch 16/60
 40/40 2s 54ms/step -
 accuracy: 0.9207 - loss: 0.2006 - val_accuracy: 0.7310 - val_loss: 0.6303
 Epoch 17/60
 40/40 2s 59ms/step -
 accuracy: 0.9247 - loss: 0.1910 - val_accuracy: 0.7247 - val_loss: 0.6617
 Epoch 18/60
 40/40 3s 62ms/step -
 accuracy: 0.9118 - loss: 0.2084 - val_accuracy: 0.7215 - val_loss: 0.7326
 Epoch 19/60
 40/40 2s 56ms/step -
 accuracy: 0.9385 - loss: 0.1648 - val_accuracy: 0.7405 - val_loss: 0.7213
 Epoch 20/60
 40/40 2s 58ms/step -
 accuracy: 0.9488 - loss: 0.1410 - val_accuracy: 0.7437 - val_loss: 0.8429
 Epoch 21/60
 40/40 3s 65ms/step -
 accuracy: 0.9387 - loss: 0.1461 - val_accuracy: 0.7595 - val_loss: 0.8566
 Epoch 22/60
 40/40 2s 50ms/step -
 accuracy: 0.9632 - loss: 0.0923 - val_accuracy: 0.7120 - val_loss: 1.0006
 Epoch 23/60
 40/40 2s 47ms/step -
 accuracy: 0.9597 - loss: 0.1038 - val_accuracy: 0.7152 - val_loss: 1.0252
 Epoch 24/60
 40/40 2s 61ms/step -
 accuracy: 0.9626 - loss: 0.1007 - val_accuracy: 0.6994 - val_loss: 1.1022
 Epoch 25/60
 40/40 3s 65ms/step -
 accuracy: 0.9576 - loss: 0.1057 - val_accuracy: 0.7500 - val_loss: 0.9836
 Epoch 26/60
 40/40 2s 54ms/step -
 accuracy: 0.9767 - loss: 0.0696 - val_accuracy: 0.7532 - val_loss: 0.9358
 Epoch 27/60
 40/40 3s 62ms/step -
 accuracy: 0.9745 - loss: 0.0821 - val_accuracy: 0.7310 - val_loss: 0.9983
 Epoch 28/60
 40/40 3s 81ms/step -
 accuracy: 0.9775 - loss: 0.0632 - val_accuracy: 0.7120 - val_loss: 1.0602
 Epoch 29/60
 40/40 4s 96ms/step -
 accuracy: 0.9776 - loss: 0.0730 - val_accuracy: 0.6962 - val_loss: 1.0660
 Epoch 30/60
 40/40 4s 95ms/step -

accuracy: 0.9747 - loss: 0.0778 - val_accuracy: 0.7310 - val_loss: 1.1709
 Epoch 31/60
 40/40 5s 83ms/step -
 accuracy: 0.9760 - loss: 0.0643 - val_accuracy: 0.7468 - val_loss: 1.0766
 Epoch 32/60
 40/40 4s 90ms/step -
 accuracy: 0.9789 - loss: 0.0588 - val_accuracy: 0.7278 - val_loss: 1.1278
 Epoch 33/60
 40/40 3s 73ms/step -
 accuracy: 0.9928 - loss: 0.0270 - val_accuracy: 0.7278 - val_loss: 1.2055
 Epoch 34/60
 40/40 3s 76ms/step -
 accuracy: 0.9854 - loss: 0.0358 - val_accuracy: 0.7342 - val_loss: 1.1635
 Epoch 35/60
 40/40 3s 75ms/step -
 accuracy: 0.9864 - loss: 0.0326 - val_accuracy: 0.7437 - val_loss: 1.3235
 Epoch 36/60
 40/40 5s 67ms/step -
 accuracy: 0.9892 - loss: 0.0284 - val_accuracy: 0.7247 - val_loss: 1.1634
 Epoch 37/60
 40/40 3s 80ms/step -
 accuracy: 0.9888 - loss: 0.0365 - val_accuracy: 0.7278 - val_loss: 1.2461
 Epoch 38/60
 40/40 3s 65ms/step -
 accuracy: 0.9939 - loss: 0.0188 - val_accuracy: 0.7184 - val_loss: 1.2304
 Epoch 39/60
 40/40 3s 81ms/step -
 accuracy: 0.9874 - loss: 0.0380 - val_accuracy: 0.7342 - val_loss: 1.1677
 Epoch 40/60
 40/40 5s 80ms/step -
 accuracy: 0.9849 - loss: 0.0534 - val_accuracy: 0.7405 - val_loss: 1.1946
 Epoch 41/60
 40/40 2s 54ms/step -
 accuracy: 0.9898 - loss: 0.0318 - val_accuracy: 0.7247 - val_loss: 1.2155
 Epoch 42/60
 40/40 2s 60ms/step -
 accuracy: 0.9945 - loss: 0.0241 - val_accuracy: 0.7405 - val_loss: 1.4959
 Epoch 43/60
 40/40 2s 60ms/step -
 accuracy: 0.9948 - loss: 0.0164 - val_accuracy: 0.7342 - val_loss: 1.3530
 Epoch 44/60
 40/40 3s 68ms/step -
 accuracy: 0.9940 - loss: 0.0183 - val_accuracy: 0.7437 - val_loss: 1.3180
 Epoch 45/60
 40/40 2s 60ms/step -
 accuracy: 0.9901 - loss: 0.0165 - val_accuracy: 0.7373 - val_loss: 1.2587
 Epoch 46/60
 40/40 2s 51ms/step -

```

accuracy: 0.9944 - loss: 0.0181 - val_accuracy: 0.7595 - val_loss: 1.3187
Epoch 47/60
40/40          2s 57ms/step -
accuracy: 0.9910 - loss: 0.0309 - val_accuracy: 0.7563 - val_loss: 1.2682
Epoch 48/60
40/40          3s 68ms/step -
accuracy: 0.9861 - loss: 0.0330 - val_accuracy: 0.7310 - val_loss: 1.3584
Epoch 49/60
40/40          2s 55ms/step -
accuracy: 0.9890 - loss: 0.0316 - val_accuracy: 0.7468 - val_loss: 1.2564
Epoch 50/60
40/40          2s 52ms/step -
accuracy: 0.9928 - loss: 0.0168 - val_accuracy: 0.7057 - val_loss: 1.3415
Epoch 51/60
40/40          3s 70ms/step -
accuracy: 0.9862 - loss: 0.0401 - val_accuracy: 0.7342 - val_loss: 1.2702
Epoch 52/60
40/40          2s 59ms/step -
accuracy: 0.9914 - loss: 0.0276 - val_accuracy: 0.7310 - val_loss: 1.4713
Epoch 53/60
40/40          2s 56ms/step -
accuracy: 0.9848 - loss: 0.0345 - val_accuracy: 0.7468 - val_loss: 1.5136
Epoch 54/60
40/40          3s 71ms/step -
accuracy: 0.9843 - loss: 0.0465 - val_accuracy: 0.7120 - val_loss: 1.3750
Epoch 55/60
40/40          3s 70ms/step -
accuracy: 0.9944 - loss: 0.0137 - val_accuracy: 0.7437 - val_loss: 1.4098
Epoch 56/60
40/40          3s 82ms/step -
accuracy: 0.9953 - loss: 0.0218 - val_accuracy: 0.7247 - val_loss: 1.4413
Epoch 57/60
40/40          3s 63ms/step -
accuracy: 0.9909 - loss: 0.0348 - val_accuracy: 0.7563 - val_loss: 1.4053
Epoch 58/60
40/40          2s 55ms/step -
accuracy: 0.9924 - loss: 0.0248 - val_accuracy: 0.7310 - val_loss: 1.4440
Epoch 59/60
40/40          2s 58ms/step -
accuracy: 0.9947 - loss: 0.0106 - val_accuracy: 0.7278 - val_loss: 1.5559
Epoch 60/60
40/40          3s 66ms/step -
accuracy: 0.9918 - loss: 0.0373 - val_accuracy: 0.7247 - val_loss: 1.4893

```

```

[5]: # Evaluate the model on the test set
loss, accuracy = model.evaluate(Xtest_resaped, Ytest)
print(f"Test Accuracy: {accuracy:.2%}")

```

```

print(f"Test Loss: {loss:.4f}")
#-----
Ypred = model.predict(Xtest_reshaped)
Ypred_classes = (Ypred > 0.5).astype(int)
Ypred_classes_flat = Ypred_classes.flatten()

# Print classification report (precision, recall, F1-score)
print("\nClassification Report:")
print(classification_report(Ytest, Ypred_classes_flat)) # Use Ytest directly
#-----
# Plot accuracy
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()

# Plot loss
plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Model Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()

plt.tight_layout()
plt.show()
#-----
roc_auc = roc_auc_score(Ytest, Ypred[:, 0])
print(f"\nROC AUC: {roc_auc:.4f}")

# Plot ROC curve
# Use Ytest for true labels and Ypred[:, 0] for predicted probabilities of the
# positive class
fpr, tpr, thresholds = roc_curve(Ytest, Ypred[:, 0])
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color='darkorange', lw=2, label='ROC curve (area = %0.2f)' %
# roc_auc)
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')

```

```

plt.title('Receiver Operating Characteristic')
plt.legend(loc="lower right")
plt.show()

#-----

cm = confusion_matrix(Ytest, Ypred_classes_flat)
plt.figure(figsize=(8, 6))
class_labels = ['Class 0', 'Class 1'] # Or provide your actual class names if
↳available

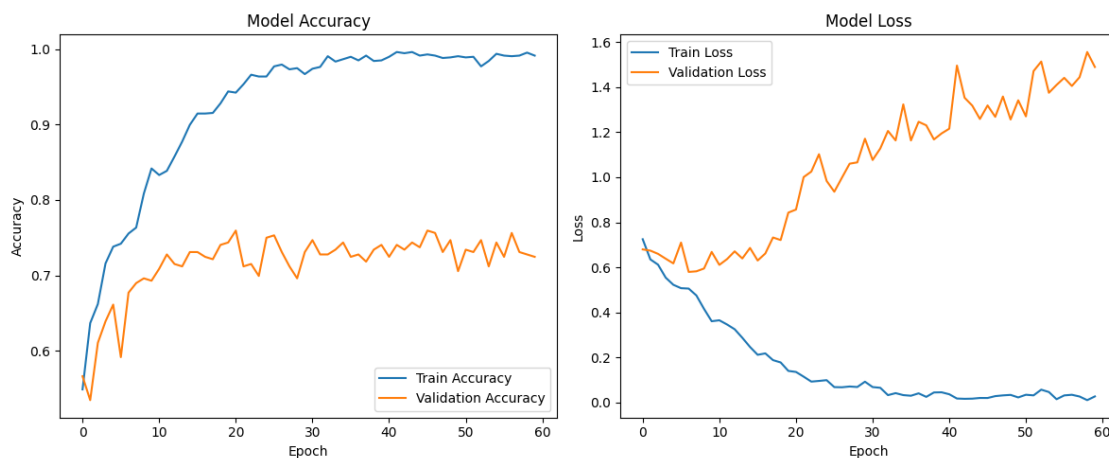
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=class_labels,
            yticklabels=class_labels)
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix')
plt.show()

```

13/13 0s 17ms/step -
accuracy: 0.7427 - loss: 1.3477
Test Accuracy: 72.91%
Test Loss: 1.3771
13/13 1s 32ms/step

Classification Report:

	precision	recall	f1-score	support
0	0.70	0.76	0.73	190
1	0.76	0.70	0.73	205
accuracy			0.73	395
macro avg	0.73	0.73	0.73	395
weighted avg	0.73	0.73	0.73	395



ROC AUC: 0.8143

